

TP 8

Équations Différentielles

ZFAI

Exercice 1 (Test préliminaire). On s'assure de la bonne importation des bibliothèques `numpy` et `matplotlib` en exécutant le programme suivant

```
import numpy as np
import matplotlib.pyplot as plt
x=np.linspace(0,2*np.pi,1000)
y=np.sin(x)
plt.plot(x,y)
plt.show()
```

Problème 1 (Le système proies-prédateurs de Volterra-Lotka). On veut simuler les équations suivantes :

$$\begin{cases} x'(t) = ax(t) - bx(t)y(t) \\ y'(t) = -cy(t) + dx(t)y(t) \end{cases}$$

en partant de $(x(0), y(0)) = (x_0, y_0)$. On prendra $a = 3$, $b = 2$, $c = 2$, $d = 3$, et $x_0 = \frac{2}{3}$, $y_0 = \frac{3}{4}$.

1. On note $X(t) = (x(t), y(t))$ la solution et $X'(t) = F(X(t))$ le système d'équations différentielles ci-dessus. La commande `np.array([x,y])` permet de produire le tableau $[x, y]$.
Écrire la fonction `FVL(X)` qui prend en argument $X = \text{array}([x, y])$ et renvoie le couple $(ax - bxy, -cy + dxy)$ en format `np.array`.
2. La fonction suivante applique le schéma d'Euler de façon générique pour n'importe quelle fonction F

```
def Euler(F,X0,T,N):
    h=T/N
    n=np.size(X0)
    S=np.zeros((n,N+1))
    S[:,0]=X0
    Xn=X0
    for i in range(N):
        Xn+=h*F(Xn)
        S[:,i+1]=Xn
    return S
```

La tester avec la fonction du modèle proies-prédateurs `FVL`, $X_0 = (\frac{2}{3}, \frac{3}{4})$, $T = 3$ et $N = 100$.

3. Tracer les résultats sous deux formes : évolution de $x(t)$ et de $y(t)$ en fonction du temps t , et évolution de $(x(t), y(t))$ dans l'espace des phases (x, y) .
4. Implémenter le schéma de Runge-Kutta d'ordre 2 : $X_{n+1} = X_n + hF(X_n + \frac{h}{2}F(X_n))$ sous forme de fonction générique `RK2(F,X0,T,N)`. Le tester.

- Implémenter le schéma d'Adams-Bashforth d'ordre 2 : $X_{n+1} = X_n + \frac{h}{2} (3F(X_n) - F(X_{n-1}))$, sous forme de fonction générique `AB2(F,X0,T,N)`. On pourra faire un premier pas avec RK2. Le tester.
- Implémenter le schéma de Runge-Kutta d'ordre 4 :

$$\begin{cases} k_0 = F(X_n), \\ k_1 = F(X_n + \frac{h}{2}k_0), \\ k_2 = F(X_n + \frac{h}{2}k_1), \\ k_3 = F(X_n + hk_2), \\ k = \frac{k_0}{6} + \frac{k_1}{3} + \frac{k_2}{3} + \frac{k_3}{6}, \quad X_{n+1} = X_n + hk \end{cases}.$$

sous forme de fonction générique `RK4(F,X0,T,N)`. Le tester.

- L'algorithme de Newton permet de résoudre les équations non-linéaires du type $G(X) = 0$ en faisant : $X^0 = X_n$ et $X^{k+1} = X^k - DG(X^k)^{-1} \cdot G(X^k)$. Implémenter le schéma d'Euler implicite : $X_{n+1} = X_n + hF(X_{n+1})$ qu'on appellera sous la forme `IEuler(F,DF,X0,T,N)` où `DF` est la matrice jacobienne de `F` c'est-à-dire la matrice de ses dérivées. On pourra utiliser le solveur de systèmes linéaires `np.linalg.solve(np.array([[...],..., [...]]), np.array([...]))` et de la matrice identité `np.eye(n)` de `numpy`.
- Implémenter et tester le schéma de Crank-Nicholson : $X_{n+1} = X_n + \frac{h}{2}(F(X_n) + F(X_{n+1}))$ qu'on appellera sous la forme `CN(F,DF,X0,T,N)`.
- Pour ces différents schémas (explicites et implicites), tracer l'intégrale première :

$$H(x, y) = dx - c \ln x + by - a \ln y.$$

Pour aller plus loin

Problème 2 (Attracteur étrange de Lorenz). On s'intéresse au système d'équations différentielles suivant :

$$\begin{cases} y_1'(t) = \sigma(y_2(t) - y_1(t)) \\ y_2'(t) = ry_1(t) - y_2(t) - y_1(t)y_3(t) \\ y_3'(t) = y_1(t)y_2(t) - by_3(t) \end{cases},$$

avec la condition initiale $(y_1(0), y_2(0), y_3(0)) = (1, 0, 0)$. On prendra $\sigma = 10$, $b = \frac{8}{3}$ et $r = 28$.

- Implémenter et tracer en fonction du temps et dans l'espace des phases (y_1, y_3) l'évolution de la solution numérique $(y_1(t), y_2(t), y_3(t))$ de ce système pour les schémas d'Euler explicite, Runge-Kutta 2 et Runge-Kutta 4 pour $0 \leq t \leq 100$.
- Faire légèrement varier (de 10^{-8} environ) :
 - la condition initiale $(y_1(0), y_2(0), y_3(0))$,
 - σ ,
 - le pas de temps h ,
 pour ces différents schémas. Tracer les résultats et comparer.